

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчет

по лабораторной работе №2

по дисциплине «Тестирование ПО»

Выполнил: Игнатов К.О., группа Р3319

Преподаватель: Тюрин И. Н.

Санкт-Петербург, 2026 г.

Содержание

1	Описание задания	2
2	Математическая модель и архитектура	2
2.1	Реализация базовых функций	2
2.2	Архитектура приложения	2
3	Стратегия тестирования и интеграция	3
3.1	Анализ классов эквивалентности	3
3.2	Интеграционное тестирование (Bottom-Up)	3
4	Логирование результатов	3
5	Сборка и запуск	3
6	Вывод	3

1 Описание задания

В рамках лабораторной работы необходимо провести интеграционное тестирование программы, осуществляющей вычисление системы функций (вариант 9311).

Правила выполнения работы:

1. Базовые функции $\sin(x)$ и $\ln(x)$ должны быть реализованы при помощи разложения в ряд Маклорена/Тейлора с задаваемой погрешностью. Использовать стандартные математические библиотеки (`Math.sin`, `Math.log`) для расчетов запрещено.
2. Производные функции должны быть выражены через базовые.
3. Для каждого модуля необходимо реализовать заглушки (Mocks).
4. Провести интеграцию приложения по 1 модулю (восходящее тестирование).
5. Разработанное приложение должно позволять выводить значения в CSV файл.

2 Математическая модель и архитектура

Система функций имеет следующий вид:

$$f(x) = \begin{cases} \left(\left(\frac{\cos(x) \cdot \csc(x)}{\sin(x)} \right)^2 \cdot \sin(x) \right) + \cot^2(x), & \text{если } x \leq 0 \\ \left((\log_3^3(x) - \log_{10}(x) - \log_2^2(x))^3 \right)^2, & \text{если } x > 0 \end{cases} \quad (1)$$

2.1 Реализация базовых функций

Базовые функции реализованы через суммы бесконечных рядов до достижения заданной точности ϵ :

- **Синус:** $\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!}$. Для оптимизации вычислений аргумент предварительно приводится к периоду $[-\pi; \pi]$.
- **Натуральный логарифм:** $\ln(x) = 2 \sum_{n=0}^{\infty} \frac{1}{2n+1} \left(\frac{x-1}{x+1} \right)^{2n+1}$. Данный ряд (ряд аретангенса) выбран благодаря сходимости на всем интервале $x > 0$.

2.2 Архитектура приложения

Приложение спроектировано с использованием паттерна Dependency Injection (Внедрение зависимостей) для обеспечения возможности тестирования в изоляции.

- **Уровень 1:** Независимые атомарные модули `Sin` и `Ln`.
- **Уровень 2:** Промежуточные модули `TrigonometryModule` и `LogarithmModule`, зависящие от объектов первого уровня.
- **Уровень 3:** Главный класс `FunctionSystem`, объединяющий логику системы.

3 Стратегия тестирования и интеграция

3.1 Анализ классов эквивалентности

Пространство входных значений было разбито на два основных класса:

1. $x \leq 0$: Тригонометрическая ветвь. Включает проверку граничной точки $x = 0$ и точек разрыва функции, где $\sin(x) = 0$ ($-\pi, -2\pi$ и т.д.).
2. $x > 0$: Логарифмическая ветвь. Включает проверку критической точки $x = 1$ (где логарифм равен 0), а также штатных позитивных значений.

3.2 Интеграционное тестирование (Bottom-Up)

Интеграция проводилась по восходящей стратегии с использованием фреймворка **Mockito**.

- **Шаг 1:** Тестирование **FunctionSystem** с полной изоляцией от реальной логики вычислений. В систему передавались заглушки (Mocks) модулей 2-го уровня. Для отключения строгих проверок неиспользуемых вызовов в Mockito применена аннотация `@MockitoSettings(strictness = Strictness.LENIENT)`.
- **Шаг 2:** Частичная интеграция. В модули 2-го уровня передавались реальные объекты базовых функций.
- **Шаг 3:** Сквозное тестирование полностью собранной системы с проверкой результатов относительно эталонных значений (WolframAlpha) с учетом дельты $\varepsilon = 0.1$.

4 Логирование результатов

Для выполнения требований по выгрузке данных реализован утилитный класс **CsvLogger**. Он принимает на вход экземпляр любого наследника **AbstractFunction**, границы вычислений и шаг. Метод генерирует файл **results.csv**, который может быть использован для дальнейшего построения графиков и ручной верификации.

5 Сборка и запуск

Сборка проекта, управление зависимостями и выполнение тестов осуществляется с помощью системы **Gradle**.

- Фреймворк тестирования: **JUnit 5 (Jupiter)**.
- Запуск тестов: `./gradlew test`.
- Запуск приложения (генерация CSV): `./gradlew run`.

6 Вывод

В ходе лабораторной работы было успешно разработано и протестировано приложение для расчета системы сложных функций. Применение внедрения зависимостей (DI) и фреймворка Mockito позволило провести грамотное интеграционное тестирование по 1 модулю, по восходящей стратегии.